



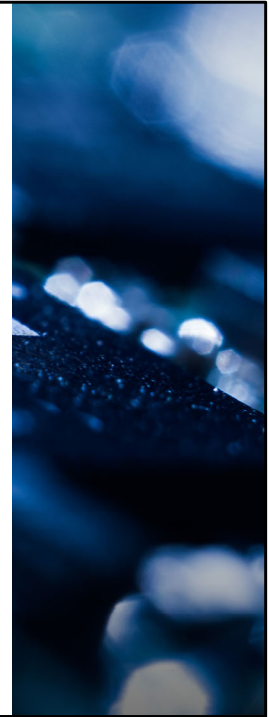
# SC1006

## Computer Organisation and Architecture

### Non- Volatile Memory

---

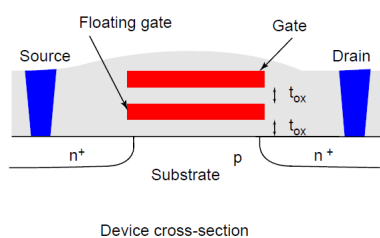
Prof Lin WeiSi  
College of Computing and Data Science, NTU  
Email: [wslin@ntu.edu.sg](mailto:wslin@ntu.edu.sg)



- This section is an introduction on **Non- Volatile Memory**

# EPROM and EEPROM

- EPROM (Erasable Programmable ROM)
  - Earliest floating gate transistors are implemented as Erasable Programmable ROM (EPROM) devices.
  - Need to put device under ultra-violet (UV) light to **erase** the stored program.



- EEPROM (Electrically Erasable PROM)
  - Advancement in process technologies makes it possible to **reduce** the **oxide thickness** ( $t_{ox}$ ).
  - Can **electrically program or erase devices**.
  - Hence, named Electrically Erasable Programmable ROM (E<sup>2</sup>PROM).

Two common non-volatile memories you can find in the market are EPROM and EEPROM

Which are short form for

Erasable Programmable ROM and  
Electrically Erasable PROM respectively

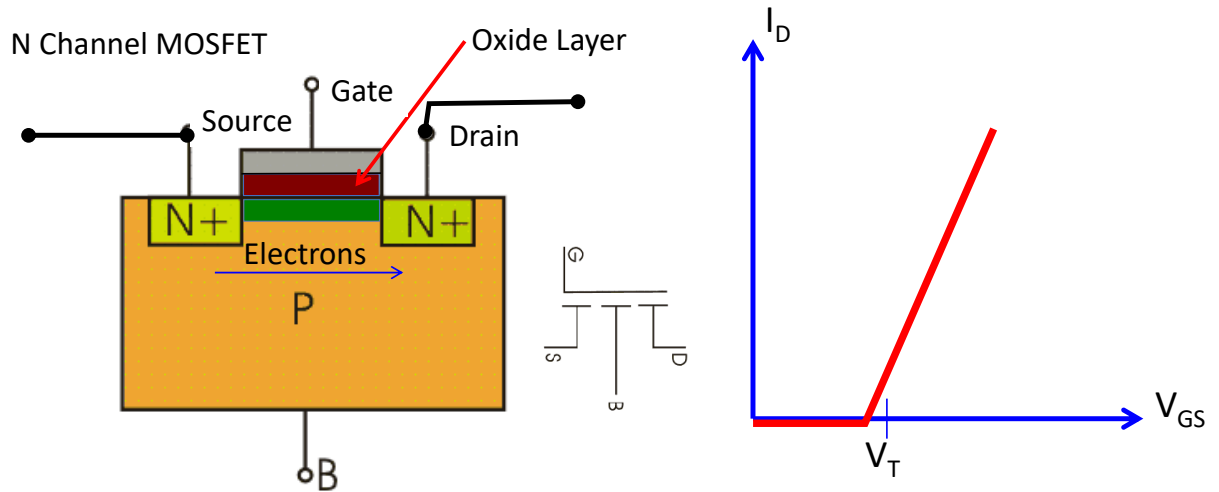
EPROM is one of the earliest erasable non-volatile memory that uses a floating gate design.

This device can be programmed electrically but requires UV light to erase its data. The packaging has a small transparent window that allows UV to get to the floating gate to remove the charges trapped there.

As technology advances, we can reduce the thickness of the gate oxide. This led to the creation of EEPROM, which allows the charges to be erased electrically.

We will talk about what is a floating gate next.

# A floating gate in MOSFET



- MOSFET (Metal-Oxide-Semiconductor Field Effect Transistor)
- For N-Channel MOSFET, if Gate-Source Voltage ( $V_{GS}$ ) > Threshold Voltage ( $V_t$ ), a conductive channel of electrons (inversion layer) will be formed and current will flow if a positive voltage is applied across Drain and Source.

So what is a floating gate?

To know that, let's start by looking at a MOSFET, or Metal-Oxide-Semicon Field Effect transistor.

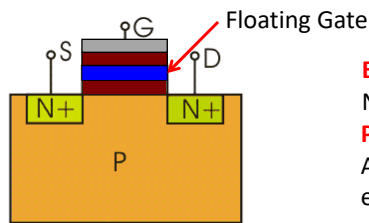
For a N-channel MOSFET, it remains non-conductive until the Gate-Source voltage exceeds a certain positive voltage threshold.

When the gate voltage is sufficiently positive, it will be able to attract more electrons which are negatively charged.

A layer of electrons will be formed under the oxide and current will flow between the Source and Drain terminal of the MOSFET, the electron layer is illustrated in this slide by a green bar below the oxide layer.

You can see from the graph that current is zero till  $V_{GS}$  exceed  $V_t$ , After which current will flow.

# Floating Gate Transistor (FGT)



## Erased State

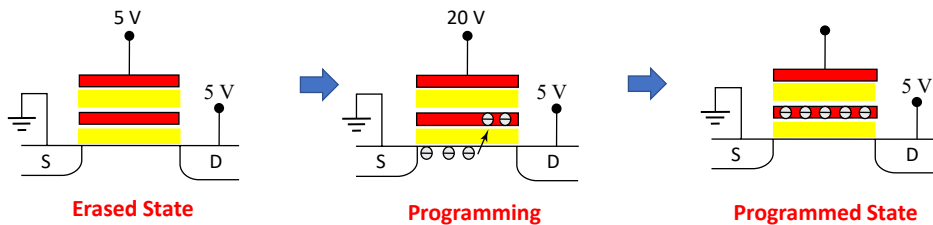
No Excess Electrons in Floating Gate

## Programming

A large positive voltage applied at the gate, causing excess electrons to tunnel through the oxide and embed in the floating gate

## Programmed State

Excess Electrons are trapped in Floating Gate even after power off



Now that you know how a MOSFET works, let's look at the floating gate transistor, FGT in short.

In the FGT, an additional layer of the metal gate and the oxide is inserted. So the gate that is sandwiched between the two oxide layers appears to be floating as it is not electrically connected to anywhere.

There are two states that the FGT may be in.

One is the erased state.

This corresponds to the state where there are no excess electrons in the FGT, i.e. it is electrically neutral.

In order to program the FGT, a large positive voltage e.g. 20V is applied to the gate.

This will cause some electrons in the substrate of the FGT to tunnel through the oxide and embed into the floating gate.

Note that the voltage value used in this slide are for illustration purpose only, actual value may differ depending on the semiconductor process.

After programming, the FGT is in the programmed state and the excess electrons in the floating gate will continue to be trapped there even if the power is removed.

The excess electrons in the floating gate has effect on the threshold voltage of the FGT and this attribute can be used to implement memory bit cell with non-volatile property. More details in the next slide.

Then there is the Programmed State

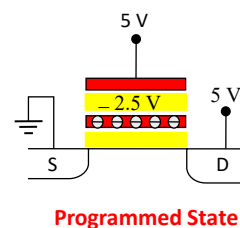
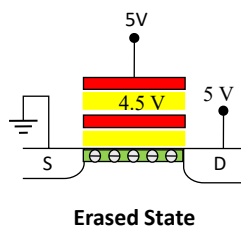
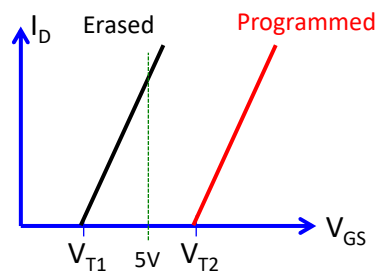
Which is the state when there are excess electrons in the FGT.

## FGT Threshold Voltage

As an illustration, let's say a 5V input at the Gate of a FGT in erased state is sufficient to turn ON the FGT.

"Programming" increases the threshold voltage of Floating Gate Transistor so the same 5V is now unable to turn ON the FGT.

Voltage values used are for illustration purpose only.



First, let's look at how the presence of excess electrons affects the threshold voltage of the FGT.

Remember that a MOSFET becomes conductive when the gate source voltage or  $V_{GS}$  is larger than a certain threshold voltage  $V_t$ .

Similarly, for FGT, It'll be conductive when its gate voltage crosses a certain threshold.

As an illustration, let's say that when the FGT is in erased state, i.e. the floating gate has no excess electrons, a 5V input at the gate can attract enough electrons to form the conductive layer. So the FGT is turned ON.

Now, if the FGT is in the programmed state instead,

Which means that excess electrons are present in the floating gate, and these negatively charged electrons will lower the resultant voltage potential, meaning user now must apply a larger voltage in order to attract the same number of electrons to form the conductive layer.

Applying the same 5V will therefore not be able to do the job, as illustrated in the diagram, the same 5V input at the gate will result in a much lower voltage potential at the floating gate when the FGT is in programmed state. This as mentioned, is due

to effect of the negative charges of the excess electrons.

So, to summarise, FGT in erase state has a lower  $V_t$  than when it is in programmed state.

This attribute of the FGT can be used to implement a memory element.

We have previously discussed about the way to detect whether a FGT is in erased or programmed state.

That can be done by injecting a voltage between the two threshold,  $V_{T1}$  and  $V_{T2}$  in the graph.

If the FGT is ON, it's in erased state,; if it is OFF, it is in programmed state.

If we further defined that the FGT stores a Logic '1' and Logic '0' when it is in erased and programmed state respectively,

then we have a 1-bit memory cell in the form of the FGT.

- To write a '1', we erase the FGT

- To write a '0', we program the FGT.

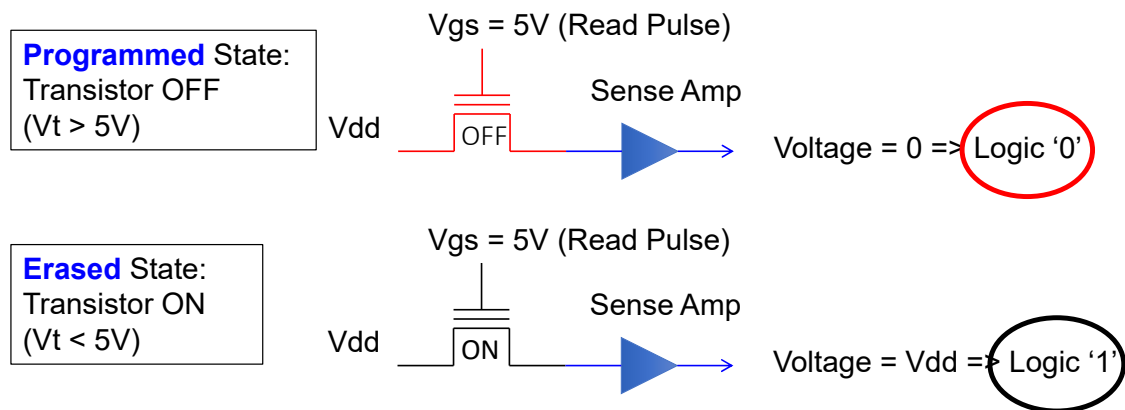
- To read the content, we apply a 5V.

Note again that the 5V here and other values are for illustration purpose only.

This concept forms the basis of many of the non-volatile memory discussed in this course, e.g. EPROM, EEPROM and Flash.

## Reading of Stored Data in Floating Gate

- With a positive Gate Voltage ( $V_{gs}$ ) = 5V
  - Transistor **OFF** if floating gate is **programmed** (contains charges).
  - Transistor **ON** if floating gate is **erased** (no charges).
- 5V value below is for **illustration only**. Actual  $V_t$  value in real world may vary depending on the doping of the transistors.



Just to recap from the discussion in the previous slide,

With a positive gate voltage, e.g. 5V, you find that FGT will remain OFF if it is in the programmed state.

Since FGT is off, the sense amplifier will detect a logic '0'.

But if the FGT is in erased state, the threshold voltage will be lower and the FGT will be turned ON, leading to sensing of logic '1' at the output.

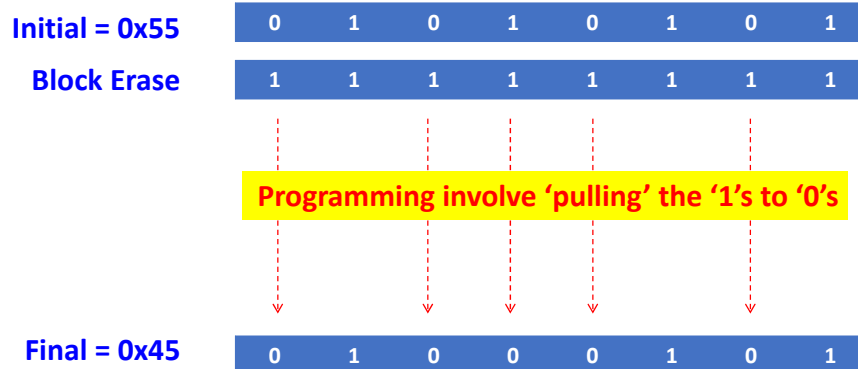
Which is why, after a flash is erased, it'll contain all '1's in its memory. During programming, we are actually storing '0's into the various memory bits.

Note again that the 5V value here is just for illustration, actual  $V_t$  value may change depending on the doping of transistors.



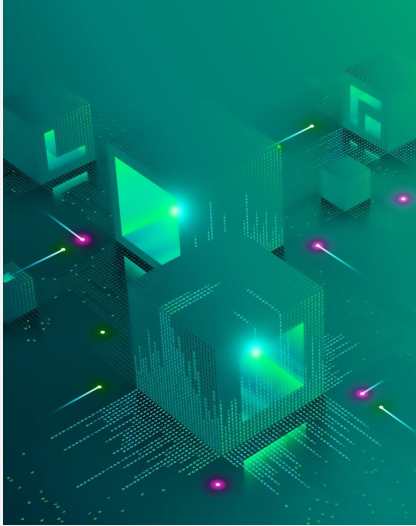
### Example of Programming a FGT based memory

- Flash '**programming**' can only modify the **cell content from '1' to '0'**
- To modify the **cell content from '0' to '1'**, an **erase operation** has to be done at block level
- Example: To Program a value of '**0x45**' when initial value in flash memory is '**0x55**'



- This slide runs through the process of programming a number into a FGT based memory such as Flash memory.
- Programming in FGT based memory is really modifying the cell content from '1' to '0'.
- If you need to write a '1' instead, that correspond to the erase operation.
- In FGT based memory, erasure are done at block level, where one block consist of multiple bytes.
- For example, if we want to write a value '0x45' to a particular memory location whose initial value was '0x55' in a Flash memory, we first have a erased the block in which the byte location is.
- This will set all the bits to '1's.
- After which, the actual programming involves setting to some of the '1's to '0's to form the value 0x45.

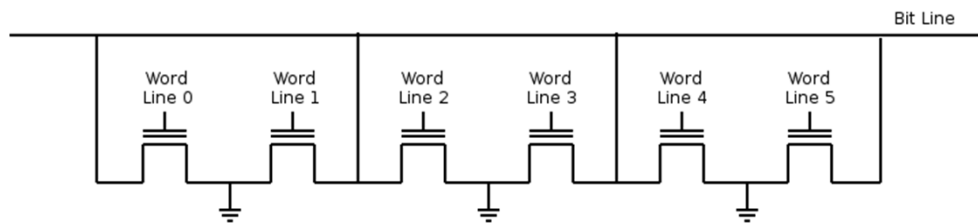
# Flash



- Based on similar floating gate technology as EEPROM.
  - Can be erased in larger blocks size compared to EEPROM (which typically has smaller page/block size).
  - Flash memory has a faster speed than EEPROM when performing write operations for large block of data.
  - Flash also cost less than EEPROM.
  - Suitable for system requiring large amount of non-volatile memory.
  - Two main types of Flash in the market
    - NAND Flash
    - NOR Flash
- 
- One of the most popular non volatile memory based on FGT is the Flash Memory.
  - Flash memory can be erased in larger block size compared to EEPROM (EPROM is no longer in use).
  - Flash memory has a faster access speed compared to EEPROM, especially when writing large blocks of data.
  - Flash memory also cost less since each overhead module for erasing or programming cater for a larger block of memory.
  - There are two main types of Flash Memory in the market today, namely the NOR and NAND flash. We will discuss in more details in later slides.

## NOR Flash

- Cell connects like a NOR gate.  
When any one of the Word Line  $> V_t(\text{Prog})$ , Bit Line output = 0.
- Supports Execute in Place, i.e. Program code stored in Parallel NOR Flash can be executed directly without the need to transfer to internal RAM first.
- Allows Random Reading of memory data using only Address information (no additional Commands needed).
- Need Special Commands (issue in Write mode) in order to perform operations other than Data Read. E.g. Program, Erase etc.
- Allows random word/byte programming. But erasure has to be done at block level. Typical Block size: 64KByte, 128KByte, 256KByte
- Use as system memory to store program code or general storage memory



In NOR flash, individual cells are connected with logic similar to that of NOR gates. Meaning that if any of the word line is higher than the threshold voltage, i.e., for erased state, the bit line output will be LOW.

Similar to the NOR gate property that if any of the input is a '1', then output will be a '0'.

NOR flash supports Execute-In-Place, XIP in short: allows code to be executed directly from flash memory without loading the code into RAM.

This means that you can store your program code in the NOR flash and execute the instruction directly from there without having to transfer the code to RAM.

This is possible because the NOR flash allows random reading of its data using only Address information.

Recall that in a processor, it only uses the Program Counter as the pointer to access instruction code, and no other commands are involved.

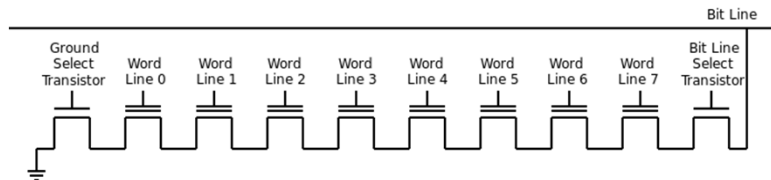
Note that this discussion refers only to NOR Read operation.

For other operations such as write operation, the procedure is more complicated and will require special commands and multiple step operations, as explained in the slide earlier for “Example of Programming a FGT based memory”.

Note that even though NOR flash allows random word or byte programming, erase operation needs to be done at block level.

Lastly, its ability to support XIP enables NOR Flash to be used as the system memory of a product. It could still be as a storage memory if user wants to.

# NAND Flash



- Cell behaves like NAND gate.
  - Bit line output = '0' only when ALL Word Line >  $V_t(\text{Prog})$ .
- Does not support execute in place operation.
  - Data has to be accessed one page at a time.
  - Command issued to open a particular page, followed by which byte(s) is/are needed in the page.
  - NAND chips uses a single bus to carry Address and Data.
- Lower Cost per Bit than NOR. Used mainly as Main Storage Memory.
  - On Board Main Storage, USB Flash Drive, SSD etc

Similarly, Nand flash got its name from the fact that the behaviour of its design has a lot similarity with the property of a NAND gate. The Bit line output is '0' only when all Word Line input >  $V_t$ .

Unlike NOR Flash, NAND flash does not support XIP.

Data is accessed at page level. Each page consists of multiple bytes of data, and multiple pages formed a block.

NAND flash uses a single IO bus to transfer Command, Address and Data information.

It's a multi-step process where the Processor will first need to send a COMMAND to the NAND Flash telling the memory what operation will be carried out, e.g. Read. It then followed up transferring the Address information and lastly the target data will be transferred.

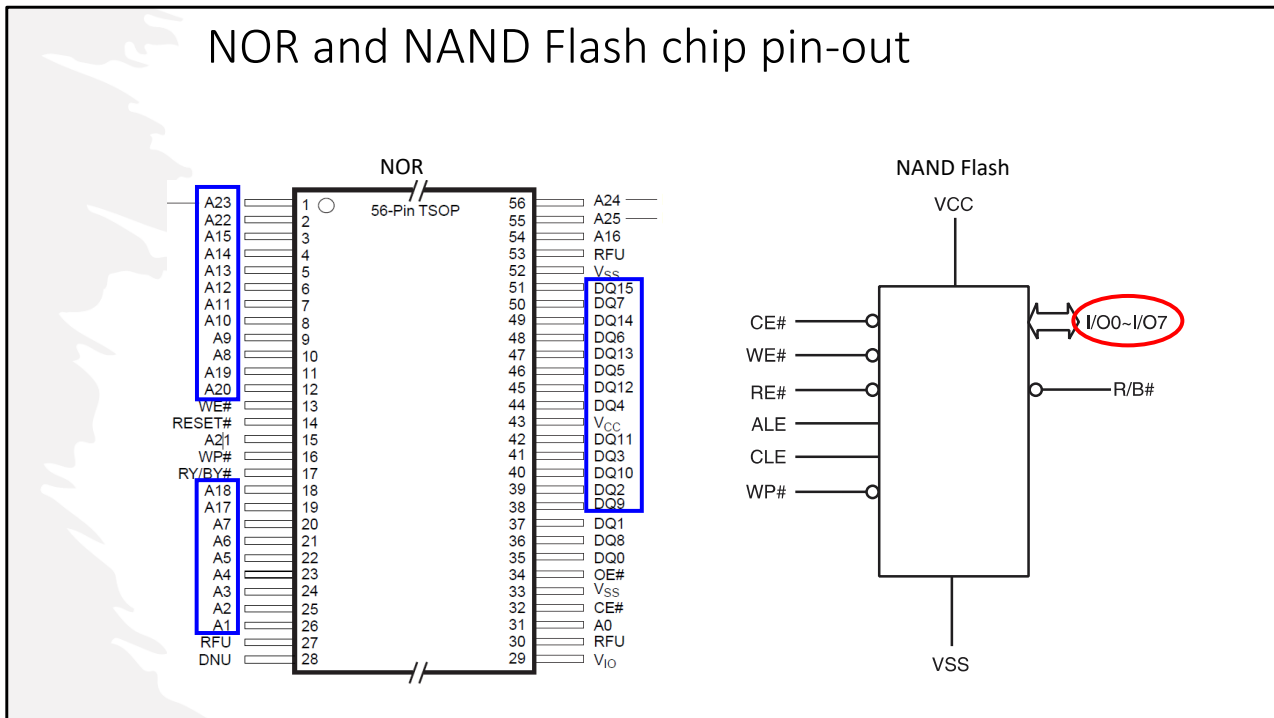
As seen, the process is much more complicated does not support XIP.

NAND flash's layout, however, has less connecting wires and therefore can be compact to a higher density, compared to NOR Flash.

This lower cost per bit advantage result in NAND flash being the memory of choice for mass storage devices.

Examples include USB Flash Drive, SSD, Phones and Tablets.

## NOR and NAND Flash chip pin-out

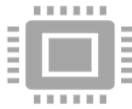


This slide illustrate the difference between the NOR and NAND Flash interface.

NOR Flash has Address and Data lines. The processor provide the Address of the target location and the NOR Flash supplies the data.

NAND Flash, on the other hand, has only one IO bus so all the COMMANDS, ADDRESS and DATA information have to go through this bus.

# System vs Storage Memory



## System Memory

Used to store runtime program and code is executed directly from the system memory

This typically

- Use SRAM/DRAM or NOR Flash
- Support XIP

DRAM technically speaking does not support XIP by itself (as its access is multi-steps) but processors that has a DRAM interface will have DRAM controller that handle the necessary translation to allow execution of code directly from the DRAM.



## Storage memory

Used to store all program and data in a computer system

Cannot run code directly from storage memory, needs to be transferred to system memory before code execution

All memory types can be used as storage memory

In practice, you usually need the storage memory to be able to retain information when the power is cut off, then **non-volatile memory** has to be used.

This is the last slide of the semiconductor memory chapter.

I'll use this slide to recap the key points for System and Storage Memory.

### System memory

Store runtime program code and data that the processor use.

Note that the code is executed directly from system memory.

System memory could be internal or external memory.

For this course, there are three memory types that supports XIP and can be used to as a system memory, namely, SRAM, DRAM and NOR Flash.

DRAM, technically speaking does not support XIP as its access is also multi-steps, but typically processor that can be connected to a DRAM will have a DRAM controller module that handles the necessary translation to allow executing code directly from DRAM.



Storage memory, on the other hand, stores all the program code/data in a computer system.

However, the processor do not run code directly from the storage memory, and it needs to be transferred to the system memory for execution.

Technically speaking, all memory types can be used as storage memory.

If you need the storage memory to be able to retain information when the power is cut off, then non-volatile memory has to be used.

That comes to the end of the semiconductor memory chapter.

We will touch on the HDD and SSD in next video.

## Slide 12

---

LW(0

Only non-volatile

Lin Weisi (Prof), 2023-04-23T01:31:58.306

No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.